

HiDiTingV100 算子驱动

开发指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

本文档主要介绍算子驱动开发相关内容，包括工作原理、按场景描述接口使用方法和注意事项。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....i

1 概述.....1

2 ADP2

2.1 概述..... 2

2.2 开发指引..... 2

2.2.1 接口说明..... 2

3 DPM.....4

3.1 概述..... 4

3.2 开发指引..... 4

3.2.1 接口说明..... 4

3.2.2 用户送帧场景 5

3.3 注意事项..... 7

1 概述

本文档主要介绍算子驱动开发相关内容，重点介绍了算子驱动各个模块，以及各个模块的开发指引。

2 ADP

2.1 概述

2.2 开发指引

2.1 概述

ADP (Application Data Port) 即定点数据的输入输出模块。

- ADP 作为输入时，可以接收外部的数据，将数据输送给后一级模块。
- ADP 作为输出时，可以通过接口向外提供定点数据。

📖 说明

一个 ADP 实例只能有一种工作模式，即单独作为输入模块或单独作为输出模块。

2.2 开发指引

2.2.1 接口说明

ADP 模块实现了以下功能：

- ADP 模块初始化、去初始化功能，该功能模块提供以下 API：
 - uapi_adp_init：初始化 ADP 模块。
 - uapi_adp_deinit：去初始化 ADP 模块。
- ADP 实例管理，用于创建和销毁 ADP 实例，该功能模块提供以下 API：
 - uapi_adp_get_def_attr：获取 ADP 默认属性。
 - uapi_adp_create：创建一路 ADP。

- uapi_adp_destroy: 销毁一路 ADP。
- uapi_adp_map: 将 ADP 映射到当前进程空间, 或者将其他业务内核创建的 ADP 映射到当前业务核心。
- uapi_adp_unmap: 解除 ADP 在当前进程空间或业务内核的映射。
- uapi_adp_get_attr: 获取 ADP 属性。
- uapi_adp_set_attr: 设置 ADP 属性。
- ADP 与其他模块级联功能, 该功能模块提供以下 API:
 - uapi_adp_attach_output: 为 ADP 绑定一个输出。
 - uapi_adp_detach_output: 解除 ADP 与输出的绑定关系。
- ADP 事件上报功能, 该功能模块提供以下 API:

uapi_adp_register_event_proc: 注册 ADP 事件处理回调函数, 事件包括数据帧信息变化、获取 EOS、上报 EOS、上报 PTS、获取延时。
- ADP 数据输入输出功能, 该功能模块提供以下 API:
 - uapi_adp_query_free: 查询 ADP 中剩余空间大小。
 - uapi_adp_query_busy: 查询 ADP 中剩余数据量大小。
 - uapi_adp_get_buffer: 获取一块 ADP buffer。
 - uapi_adp_put_buffer: 写完数据后, 更新 ADP buffer。
 - uapi_adp_send_stream: 向 ADP buffer 送入数据, 使用 stream 结构体。
 - uapi_adp_acquire_stream: 从 ADP buffer 获取数据, 使用 stream 结构体。
 - uapi_adp_release_stream: 释放 ADP buffer 中的数据, 使用 stream 结构体。
 - uapi_adp_send_frame: 向 ADP buffer 送入数据, 使用 frame 结构体。
 - uapi_adp_acquire_frame: 从 ADP buffer 获取数据, 使用 frame 结构体。
 - uapi_adp_release_frame: 释放 ADP buffer 中的数据, 使用 frame 结构体。

3 DPM

3.1 概述

3.2 开发指引

3.3 注意事项

3.1 概述

DPM (Data Process Module) 即数据处理模块。DPM 模块接收前端模块或用户发送的定点数据，根据用户选择的算子输出数据，生成的数据返回给用户使用。

DPM 对输入数据进行处理后输送给下一级模块。

说明

DPM 模块目前仅支持 FIR 算子。

3.2 开发指引

3.2.1 接口说明

DPM 模块实现了以下功能：

- DPM 模块系统管理功能，实现初始化、去初始化等功能，该功能模块提供以下 API：
 - uapi_dpm_init：初始化 DPM 模块。
 - uapi_dpm_deinit：去初始化 DPM 模块。
- DPM 模块实例管理功能，该功能模块提供以下 API：

- uapi_dpm_create: 创建一路数据处理实例。
- uapi_dpm_destroy: 销毁一路数据处理实例。
- uapi_dpm_get_attr: 获取数据处理实例的属性。
- uapi_dpm_set_attr: 设置数据处理实例的属性。
- uapi_dpm_start: 启动数据处理实例。
- uapi_dpm_stop: 停止数据处理实例。
- DPM 与其他模块级联功能, 该功能模块提供以下 API:
 - uapi_dpm_attach_output: 为数据处理实例绑定一个输出。
 - uapi_dpm_detach_output: 为数据处理实例解绑一个输出。
- DPM 事件上报功能, 该功能模块提供以下 API:
 - uapi_dpm_register_event_proc: 注册 DPM 事件处理回调函数, 事件包括数据处理和数据获取时延。

算子实例属性如下:

- dpm_id: 每种 DPM 算子都唯一对应的 ID。
- param: 其他参数, 主要包括声道数、位宽、采样率、私有数据地址以及长度。

📖 说明

uapi_dpm_register_event_proc 暂不支持。

3.2.2 用户送帧场景

用户根据自己的实际需求, 将需要处理的数据通过 ADP 实例送到 DPM 模块进行处理, 并通过另一个 ADP 实例获取处理后的数据。

📖 说明

在调用其他 DPM 接口之前, 必须先对 DPM 模块初始化; 不使用 DPM 模块时, 须将 DPM 模块去初始化。

用户送帧初始化步骤

用户送帧初始化步骤如下:

步骤 1 调用 uapi_adp_init 接口初始化 ADP 模块:

```
uapi_adp_init();
```

步骤 2 调用 uapi_dpm_init 接口初始化 DPM 模块:

```
uapi_dpm_init();
```

步骤 3 调用 `uapi_adp_get_default_attr` 接口获取输出 ADP 的默认创建属性：

```
uapi_adp_attr adp_attr;  
uapi_adp_get_def_attr(&adp_attr);
```

步骤 4 调用 `uapi_adp_create` 接口创建一路输出 ADP，用于获取 DPM 的输出：

```
td_handle h_adp_out = 0;  
uapi_adp_create(&h_adp_out, &adp_attr);
```

步骤 5 调用 `uapi_dpm_create` 接口创建一路 DPM，用于数据的处理：

```
td_handle h_dpm = 0;  
uapi_dpm_create(&h_dpm);
```

步骤 6 调用 `uapi_dpm_get_attr` 接口获取 DPM 的创建属性：

```
uapi_dpm_attr dpm_attr;  
uapi_dpm_get_attr(h_dpm, &dpm_attr);
```

步骤 7 修改属性，调用 `uapi_dpm_set_attr` 接口设置新属性：

```
dpm_attr.dpm_id = UAPI_DPM_ID_FIR;  
dpm_attr.param.private_data = TD_NULL;  
dpm_attr.param.private_data_size = 0;  
uapi_dpm_set_attr(h_dpm, &dpm_attr);
```

步骤 8 调用 `uapi_dpm_attach_output` 接口将 DPM 的输出设置为输出 ADP：

```
uapi_dpm_attach_output(h_dpm, h_adp_out);
```

步骤 9 调用 `uapi_adp_get_default_attr` 接口获取输入 ADP 的默认创建属性：

```
uapi_adp_get_def_attr(&adp_attr);
```

步骤 10 调用 `uapi_adp_create` 接口创建一路输入 ADP，用于给 DPM 送入定点数据：

```
td_handle h_adp_in = 0;  
uapi_adp_create(&h_adp_in, &adp_attr);
```

步骤 11 调用 `uapi_adp_attach_output` 接口将输入 ADP 的输出设置为 DPM：

```
uapi_adp_attach_output(h_adp_in, h_dpm);
```

步骤 12 调用 `uapi_dpm_start` 接口启动 DPM 进行处理。

```
uapi_dpm_start(h_dpm);
```

步骤 13 创建一个送数据线程，反复调用 `uapi_adp_send_stream` 接口向输入 ADP 送入数据：

```
td_s32 ret;  
td_bool task_active = TD_TRUE;
```

```
uapi_audio_stream stream;
while (task_active) {
    read_stream(&stream); /* read_stream需自行实现，用于获取数据 */
    while (task_active) {
        ret = uapi_adp_send_stream(h_adp, &stream);
        if (ret != EXT_SUCCESS) {
            continue;
        }
        break;
    }
}
```

步骤 14 创建一个接收数据线程，调用 `uapi_adp_acquire_stream` 接口从输出 ADP 获取处理后的数据，使用完之后调用 `uapi_adp_release_stream` 接口释放该数据：

```
td_s32 ret;
td_bool task_active = TD_TRUE;
uapi_stream_buf output_stream;
while (task_active) {
    while (task_active) {
        ret = uapi_adp_acquire_stream(h_adp, &output_stream);
        if (ret != EXT_SUCCESS) {
            continue;
        }
        process_stream(output_stream); /* process_stream需自行实现，用于处理处理后的数据 */
        ret = uapi_adp_release_stream(h_adp, &output_stream);
        if (ret != EXT_SUCCESS) {
            continue;
        }
        break;
    }
}
```

说明

DPM 没有定点数据接口，需要创建一路输入 ADP 和一路输出 ADP，并且将 DPM 绑定到输入 ADP 的后级，同时将输出 ADP 绑定到 DPM 的后级。

----结束

3.3 注意事项

- 绑定输入源/输出源需要在处理启动之前，取消绑定关系需要在停止处理之后进行。

- 在调用 `uapi_dpm_set_attr` 接口设置 DPM 算子模块属性前必须先停止处理。配置好处理属性之后，再启动处理。
- 在调用 `uapi_dpm_set_attr` 接口动态配置处理属性时要注意传入属性的合法性。合法性包括：各个参数在规定的取值范围内；非动态配置的属性不能予以更改。建议在调用 `uapi_dpm_set_attr` 接口前先调用 `uapi_dpm_get_attr` 接口获取到当前算子属性，再针对要修改的属性值进行修改。

Draft